Appendix E. Recursion

If you are reading this book, you are probably already familiar with loops. As discussed in appendix A, `for`, `while`, `do-while` loops (and some more, depending on the language) are all examples of iteration.

Iterative loops are one straight-forward way to repeatedly perform similar actions on a series of elements. Usually loops goes hand-in-hand with certain data structures, for example containers like lists and arrays.

Loops tend to work well when the data structure has a "linear shape"; sometimes, though, things are more complicated and a simple loop might not be enough to immediately solve the issue. We will see, for example, how trees and graphs are more challenging than lists. There are workarounds to use loops with these structures as well, but often a different approach works better.

In the rest of this book we used pseudo-code to describe algorithms, in a language-agnostic way. To better illustrate how recursion works, however, a real programming language works better than pseudo-code. So, for this appendix only, we will use JavaScript in code listings. The choice of JavaScript, among many possible candidates, is motivated by two characteristics of this language:

1. Full read/write closures
2. Functions treated as first class citizens